

Análisis léxico del lenguaje Prolog: investigación, diseño, implementación y validación

INFO1148, Teoría de la Computación

Fernando Valdes
Juan Muñoz Veloso
Vicente Rivera

17 de septiembre de 2026 — Prof. M. Lévano

Campo	Valor
Curso	INFO1148, Teoría de la Computación
Repositorio	https://github.com/FernandoValdes01/Tarea01TC

Resumen ejecutivo

Este informe presenta el análisis, diseño, implementación y validación de un analizador léxico para un subconjunto de Prolog inspirado en la notación de ISO Prolog y SWI-Prolog. El trabajo define formalmente el catálogo de tokens con una expresión regular por categoría, modela las categorías principales con autómatas finitos, muestra la determinización por construcción de subconjuntos y la minimización por refinamiento de particiones sobre un subconjunto representativo, e implementa el lexer en Python con recorrido de izquierda a derecha, máxima coincidencia y prioridades explícitas. El lexer emite tipo, lexema original, línea y columna por token, administra una tabla de lexemas con deduplicación por par tipo y lexema, y reporta seis clases de error con fragmento y posición continuando el análisis. La validación usa una suite de 85 pruebas automatizadas con 26 casos válidos, 22 de prioridad, 16 inválidos y dos archivos completos, con resultado real de 85 aprobadas y cero fallos.

Palabras clave: análisis léxico, Prolog, expresiones regulares, autómatas finitos, tabla de lexemas.

1. Introducción

El análisis léxico es la primera fase de un compilador: transforma la cadena fuente en una secuencia ordenada de tokens con posición, separando las decisiones regulares (qué formas son válidas) de las decisiones sintácticas (en qué orden pueden aparecer). Un lexer correcto y bien diagnosticado simplifica el parser, produce errores comprensibles y deja evidencia reproducible de cada decisión.

Este informe documenta el subconjunto de Prolog reconocido, su especificación formal, los autómatas que lo modelan, la implementación en Python, la tabla de lexemas, el tratamiento de errores y el plan de pruebas con resultados reales. Cada afirmación remite a su respaldo: sección de la especificación, módulo del código, prueba automatizada o salida de un comando ejecutado.

2. Objetivos

El objetivo general es analizar, diseñar, implementar y validar un analizador léxico para un subconjunto definido de Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos, con pruebas reproducibles y diagnósticos verificables.

Los objetivos específicos medibles son: definir formalmente cada categoría con nombre de token, patrón, ejemplos válidos e inválidos y atributo asociado; construir una expresión regular por categoría resolviendo superposiciones con máxima coincidencia y prioridades; modelar las categorías principales con AFN o AFD y presentar un AFD integrado o una estrategia equivalente con estados de aceptación identificados; mostrar la determinización por construcción de subconjuntos y la minimización con justificación de equivalencias sobre un subconjunto representativo; implementar el lexer con posiciones 1-based, tabla de lexemas y seis clases de error recuperables; y validar con al menos 20 pruebas válidas, 8 inválidas, casos de prioridad y dos archivos completos, registrando el resultado real de la suite.

3. Alcance del analizador

El analizador reconoce exactamente el catálogo de `docs/informe/01_especificacion_lexica.md`: átomos simples y entre comillas simples, variables nombradas y anónima, enteros y reales sin signo, strings con comillas dobles, los operadores de las seis familias (cláusula, consulta, DCG, unificación y comparación, aritméticos, control), los nueve delimitadores de un carácter y los elementos ignorables (espacios, tabulaciones, saltos de línea y comentarios de línea y de bloque).

Quedan explícitamente fuera del alcance el análisis sintáctico y semántico: no se verifica el orden de los tokens, no se construye un árbol sintáctico, no se comprueba el balance de paréntesis, corchetes o llaves, no se resuelven ámbitos ni variables repetidas, no se implementa unificación, no se evalúan expresiones y no se decide si una secuencia de tokens forma un programa válido. Una `)` aislada es un token `RPAREN` correcto; su posible improcedencia pertenece a una fase posterior.

4. Especificación léxica

El contrato completo vive en `docs/informe/01_especificacion_lexica.md` y este capítulo lo resume sin modificarlo. El lexema de cada token es el segmento original de entrada, sin normalización ni conversión de números y sin des-escapar literales. El alfabeto de identificadores es ASCII y cualquier letra, dígito o espacio Unicode fuera de un literal es un carácter no admitido.

Token	Patrón Python (re)	Válidos	Exclusiones
ATOM	<code>[a-z][a-z0-9_]*</code>	<code>padre, persona_1, isla</code>	<code>Padre, á</code>
QUOTED_ATOM	<code>'...\\['nrt]...</code>	<code>'Juan Pérez', ':-', 'it\\'s'</code>	sin cierre, <code>'a\\q'</code>
VARIABLE	<code>[A-Z]... o _[...]+</code>	<code>X, Persona, _Temporal</code>	<code>_ solo, 9X</code>
ANONYMOUS	<code>-</code>	<code>-</code>	<code>_X, __</code> (son VARIABLE)
INTEGER	<code>[0-9]+</code>	<code>0, 25, 0007</code>	<code>5., .5</code>
REAL	<code>[0-9]+\\. [0-9]+</code>	<code>3.14, 0.5</code>	<code>5., 1.2.3</code> , exponentes
STRING	<code>"...\\["nrt]...</code>	<code>"hola", "línea\\n"</code>	sin cierre, <code>"a\\q"</code>
OPERATOR	unión normativa (Sec. 8)	<code>:-, ?- , -->, \\=, =..</code>	<code>:, ?, \\</code> aisladas
Delimitadores	un carácter	<code>() [] {}, ,</code>	sin validación de balance

Las familias de `OPERATOR` son `CLAUSE` (`:-`), `QUERY` (`?-`), `DCG` (`-->`), `UNIFICATION_COMPARISON` (`=, \\=, ==, \\==, =.., <, <=, >, >=`), `ARITHMETIC` (`+, -, *, /, //, **, is, mod`) y `CONTROL` (`\\+`,

!, ;). La unión normativa se implementa como un mapa recorrido por longitud descendente. Las palabras `is` y `mod` solo son operadores cuando el lexema completo no tiene continuación de identificador.

Las convenciones adoptadas son: los números no tienen signo (-12 son dos tokens); `INTEGER` y `REAL` no admiten exponentes ni puntos sin dígitos a ambos lados; los escapes válidos son exactamente barra inversa más uno permitido según el delimitador (`\\`, `\'`, `\"`, `\n`, `\r`, `\t`), y cualquier otra pareja es `INVALID_ESCAPE`; ningún literal admite un salto de línea real en su interior; `%` inicia el comentario

5. Diseño de autómatas

Las figuras usan la convención `estado --símbolo--> estado`, con `(( ))` para aceptación y la etiqueta del token reconocido. Abreviaturas: `LOWER=[a-z]`, `UPPER=[A-Z]`, `DIGIT=[0-9]`, `ID=[A-Za-z0-9_]`.

Figura 1 (átomos): `q0 --LOWER--> ((q1 ATOM))`; `q1 --[a-z0-9_]-> q1`.

Figura 2 (variables): `q0 --UPPER--> ((q1 VARIABLE))`; `q0 --_--> q2`;

`q1 --ID--> q1`; `q2 --ID--> ((q3 VARIABLE))`; `q3 --ID--> q3`;

`q2` sin continuación = `ANONYMOUS_VARIABLE` por prioridad.

Figura 3 (números): `q0 --DIGIT--> ((q1 INTEGER))`; `q1 --DIGIT--> q1`;

`q1 --.-> q2`; `q2 --DIGIT--> ((q3 REAL))`; `q3 --DIGIT--> q3`;

`q0 --.-> q4`; `q4 --DIGIT--> q5`; `q5 --[0-9.]-> q5` (trampa).

Figura 4 (operadores): `s0` con ramas `\ -> {\=, \==, \+}`,

`= -> {=, ==, =., =<}`, `/ -> {/, //, /*}`, `* -> {*, **}`,

`:` `-> {:-}`, `?` `-> {?-}`, `-` `-> {-->}`, más `<`, `>`, `+`, `!`, `;`;

cada aceptación porta lexema y familia.

Figura 5 (ignorables): `[ \t]` en ciclo de una columna; `\n`, `\r`, `\r\n`

un salto; `%` hasta antes del salto; `/* ... */` hasta el primer `/*`

o `UNTERMINATED_BLOCK_COMMENT`.

Figura 6 (integrado): despachador por primera clase + máxima

coincidencia + prioridades; equivalente a la unión de Figuras 1-5.

El estado inicial de cada autómata es `q0` (o `s0`); los estados de aceptación y sus tokens se indican en cada figura. La trampa `q5` captura `.5` y las corridas que el validador reporta como `MALFORMED_NUMBER`. La recuperación amplía el fragmento con un sufijo contiguo de identificador, como en `12abc`, antes de validar exactamente `INTEGER` o `REAL`.

6. Determinización y minimización

Para la determinización se tomó el subconjunto representativo de ambigüedad entre operador y comentario con las ramas `/`, `//` y `/* ... */`, construido por Thompson como unión de tres ramas con transiciones vacías desde un inicio común. La ε -cerradura del inicio es el conjunto de los tres primeros estados de rama más el inicio, y como ninguna rama intermedia usa transiciones vacías adicionales, cada ε -cerradura posterior es la identidad; se declara así porque el cálculo se hizo y su resultado es trivial, no porque se haya omitido.

Subconjunto	Con /	Con *	Con otro	Aceptación
S0 (inicio)	S1	—	—	ninguna
S1 (tras /)	S2	S3	operador /	OPERATOR(' /')
S2 (tras //)	—	—	fin	OPERATOR(' //')
S3 (cuerpo /*)	S3	S4	S3	ninguna
S4 (tras *)	cierre si sigue /	S4	S3	cierre con */

La tabla muestra por qué el comentario debe intentarse antes que el operador: desde S1 el símbolo `*` compromete al autómata con el bloque y abandonar esa rama violaría la máxima coincidencia. El lexer respeta este orden al tratar `/*` en la fase de ignorables antes de probar operadores.

Para la minimización se tomó el AFD de números de la Figura 3 con la partición inicial entre aceptadores (`q1` entero, `q3` real) y no aceptadores. El refinamiento separa `q1` de `q3` porque ante `.` transitan a bloques distintos, lo que justifica que entero y real nunca se fusionen y que la decisión entre ambos la tome la longitud del lexema. Los estados trampa de número mal formado resultan equivalentes entre sí porque ciclan sobre `[0-9.]` sin aceptar, de modo que se fusionan en una única trampa que el conductor reporta como `MALFORMED_NUMBER` con la corrida completa.

7. Reglas de prioridad y máxima coincidencia

En cada posición se elige el candidato válido más largo y solo en empate se aplica la prioridad contractual.

Entrada	Candidatos	Regla	Resultado
<code>\==</code>	<code>\==</code> , <code>\=</code>	más largo	OPERATOR unificación (P01)
<code>=..</code>	<code>=..</code> , <code>=</code>	más largo	OPERATOR unificación (P02)
<code>==</code>	<code>==</code> , <code>=</code>	más largo	OPERATOR unificación (P03)
<code>:-</code>	<code>:-</code> frente a <code>:</code>	2 caracteres	OPERATOR cláusula (P05)
<code>?-</code>	<code>?-</code> frente a <code>?</code>	2 caracteres	OPERATOR consulta (P06)
<code>--></code>	<code>--></code> , <code>-</code>	más largo	OPERATOR DCG (P07)
<code>//</code>	<code>//</code> , <code>/</code>	más largo	OPERATOR aritmético (P08)
<code>**</code>	<code>**</code> , <code>*</code>	más largo	OPERATOR aritmético (P09)
<code>-</code>	anónima	regla del anónimo	ANONYMOUS (P10)
<code>_Tmp</code>	variable completa	más largo	VARIABLE (P11)
<code>isla</code>	ATOM(4) vs prefijo	guarda	ATOM (P12)
<code>is, mod</code>	empate ATOM/OPERATOR	reservada	OPERATOR aritm. (P13-14)
<code>1.25</code>	REAL vs INTEGER	más largo	REAL (P16)
<code>X=Y</code>	adyacentes	avanza cursor	3 tokens (P17)
<code>12abc</code>	candidato con sufijo inválido	hasta separador seguro	MALFORMED sin tokens (P21)
<code>5., .5, 1..2</code>	numérico inválido	detector	MALFORMED (I07-09)

El caso `12abc` aplica la recuperación exigida por el Prompt 03: se consume como un único fragmento numérico inválido y no se emiten tokens parciales.

8. Diseño e implementación

La arquitectura tiene cuatro módulos con una sola dirección de dependencias: `tokens.py` define `TokenType`, `OperatorFamily`, el mapa `OPERATOR_FAMILY` y el `Token` con posición de inicio y atributos opcionales; `errors.py` define `LexicalErrorKind` y `LexicalError`, con alias compatibles

para el corpus; `symbol_table.py` implementa `SymbolTable` con deduplicación por par tipo y lexema; `lexer.py` expone la clase `Lexer` y los adaptadores `tokenize` y `tokenize_file`; y `main.py` ofrece la CLI. La salida básica conserva `<TIPO, lexema, línea, columna>`; `--show-attributes` muestra familia o índice y `--tabla` vuelca la tabla.

El flujo recorre la entrada de izquierda a derecha: primero consume ignorables, luego procesa literales, números, identificadores, operadores por longitud descendente, delimitadores y finalmente registra `INVALID_CHARACTER`. Cada token guarda la línea y columna de su primer carácter; la tabulación avanza una columna y `\r\n` cuenta como un único salto.

La salida real del programa válido empieza con `<ATOM, 'padre', 2, 1>`, `<LPAREN, '(', 2, 6>`, `<ATOM, 'juan', 2, 7>` y totaliza 207 tokens, 0 errores y 48 entradas de tabla. Los comandos reproducibles son `python -m compileall src tests`, `python -m pytest -q`, `python -m src.main tests/corpus` y `python -m src.main tests/corpus/entradas/programa_con_errores.pl` (en este entorno, la suite se ejecutó con `uv run --with pytest`).

9. Tabla de lexemas

Se registran `ATOM`, `QUOTED_ATOM` y `VARIABLE` como identificadores, más `INTEGER`, `REAL` y `STRING` como literales conservables, siempre con el lexema original y sin conversiones. La clave de deduplicación es el par tipo y lexema: dos apariciones idénticas comparten índice y dos formas textuales distintas nunca se fusionan. Cada token registrable porta su índice puramente léxico. La variable anónima `_` nunca se registra, y tampoco operadores, delimitadores, espacios ni comentarios. La tabla no guarda alcances, usos, unificaciones ni tipos semánticos.

10. Tratamiento de errores léxicos

Los seis errores contractuales se acumulan sin detener el recorrido y cada diagnóstico incluye clase, fragmento, línea, columna y mensaje útil. El programa con errores produce exactamente estos 12 diagnósticos: `UNTERMINATED_QUOTED_ATOM` en 3:6, `UNTERMINATED_STRING` en 5:6, `INVALID_CHARACTER('@')` en 7:6, `MALFORMED_NUMBER('5.')` en 8:6, `MALFORMED_NUMBER('.5')` en 9:6, `MALFORMED_NUMBER('1..2')` en 10:6, `INVALID_CHARACTER(':')` en 11:5, `INVALID_CHARACTER('??')` en 12:5, `INVALID_CHARACTER` de barra inversa en 13:5, `INVALID_ESCAPE` en 14:8 y 15:8, y `UNTERMINATED_BLOCK_COMMENT` en 17:1 hasta EOF. El bloque sin cierre no admite tokens posteriores; todos los demás errores dejan tokens válidos después.

11. Plan de pruebas y resultados

La suite `tests/test_lexer.py` suma 85 pruebas: 3 del modelo `Token`, 26 válidas con tokens y posiciones exactas, 22 de prioridad, 16 inválidas con clase, fragmento, posición y recuperación, 12 transversales de posiciones, comentarios, escapes, tabla y acumulación, y 6 de archivos incluyendo los dos programas completos y los corpus en disco. La matriz completa con el resultado real de cada caso vive en `docs/informe/04_matriz_de_pruebas.md`.

El archivo válido produce 207 tokens, 0 errores y 48 entradas, cubriendo los 17 tipos de token y las 6 familias. El archivo con errores produce 69 tokens, 12 errores de las 6 clases en 12 líneas distintas y 28 entradas. El comando ejecutado fue `uv run --with pytest python -m pytest -q` con resultado real 85 passed; además `python -m compileall src tests` terminó sin errores y `git diff --check` salió limpio.

12. Análisis de resultados

La cobertura es total sobre el catálogo: cada tipo de token y cada familia aparece en el corpus válido, cada clase de error tiene al menos dos variantes cuando el contrato lo permite y cada regla de prioridad tiene su prueba dedicada. Los casos límite confirman que CRLF cuenta como un salto, que la tabulación avanza una columna, que los comentarios dentro de literales son contenido, que `-12` son dos tokens y que `)` aislado sigue siendo token.

La principal dificultad fue la interacción entre el punto de fin de cláusula y el detector numérico: una cláusula que termina en número sin espacio es `MALFORMED_NUMBER` por contrato, de modo que los archivos válidos separan con espacio. `12abc` se consume como un único error numérico, según exige el Prompt 03. La recuperación quedó demostrada con 69 tokens válidos alrededor de 11 errores recuperables. El único límite real es documental: `agents.md` y la plantilla de informe no se encontraron en las rutas indicadas.

13. Conclusiones

El trabajo muestra que un lexer riguroso nace de decisiones contractuales explícitas antes que del código: alfabeto ASCII, números sin signo, escapes cerrados por delimitador, comentarios no anidables y máxima coincidencia con prioridades resolvieron todas las ambigüedades sin casos especiales en la implementación. Los objetivos se cumplieron de forma medible con 85 pruebas aprobadas, cobertura total de categorías y familias, y dos archivos completos que ejercitan el recorrido real.

Las limitaciones reales son que el lexer solo reconoce el subconjunto contratado y que los archivos válidos deben separar con espacio un número del punto final. Como mejoras futuras se proponen modos de salida adicionales (JSON o CSV de tokens), un visualizador de autómatas generado desde las expresiones del código y pruebas de rendimiento sobre archivos grandes, sin ampliar el subconjunto ni entrar en análisis sintáctico.

14. Referencias

Las fuentes realmente consultadas, en formato APA 7, se detallan en `docs/informe/referencias.md` y comprenden el enunciado de la tarea, la especificación léxica contractual, la matriz de pruebas, el código y el corpus del repositorio, y la documentación oficial de Python usada para implementar y ejecutar la suite. No se inventaron editoriales, autores, URL ni fechas.

15. Anexos

Los anexos viven en `docs/informe/anexos.md` e incluyen el enlace real al repositorio, los listados de autómatas completos, las salidas extensas de ambos programas, la evidencia de los comandos de verificación y la lista de comprobación contra la rúbrica con el archivo o prueba que respalda cada dimensión.

Anexo A. Diagramas de autómatas

Analizador léxico de un subconjunto de Prolog

Fuentes editables: docs/automatas/diagramas/*.mmd

Figura 1. AFD de átomos

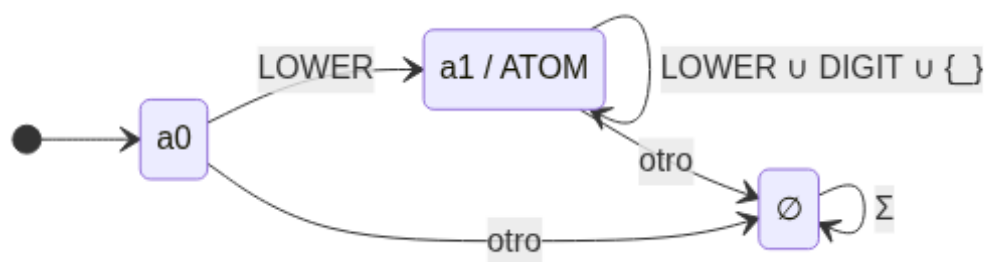


Figura 2. AFD de variables y variable anónima

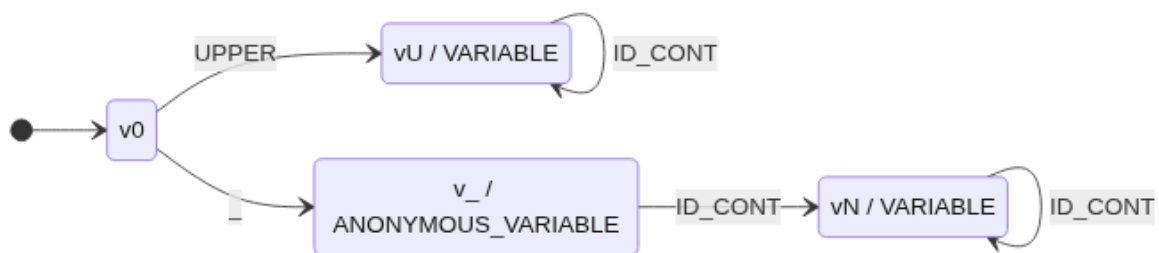


Figura 3. AFD de números

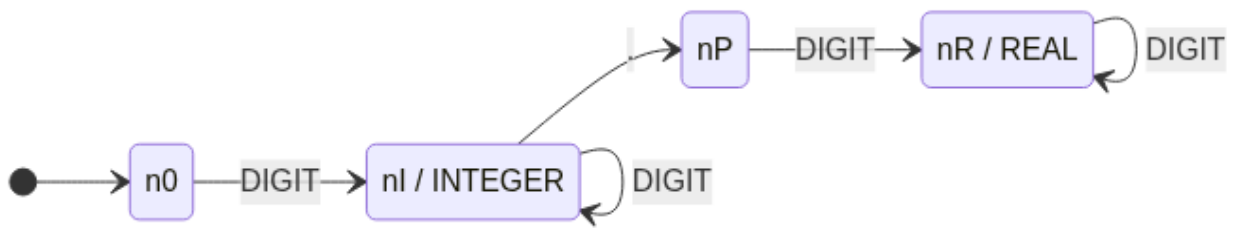


Figura 4. AFD de literales entrecomillados

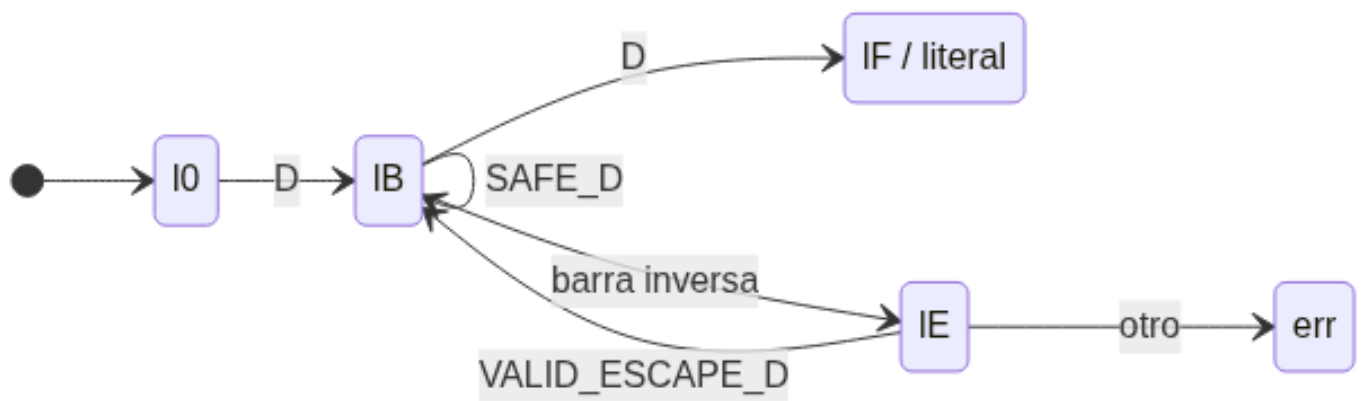


Figura 5. AFD de comentarios

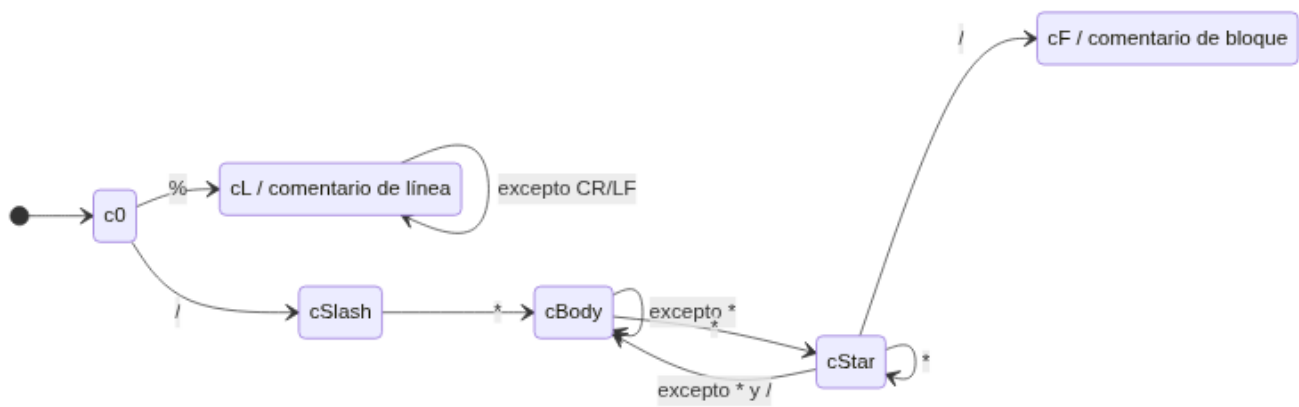


Figura 6. Trie de operadores

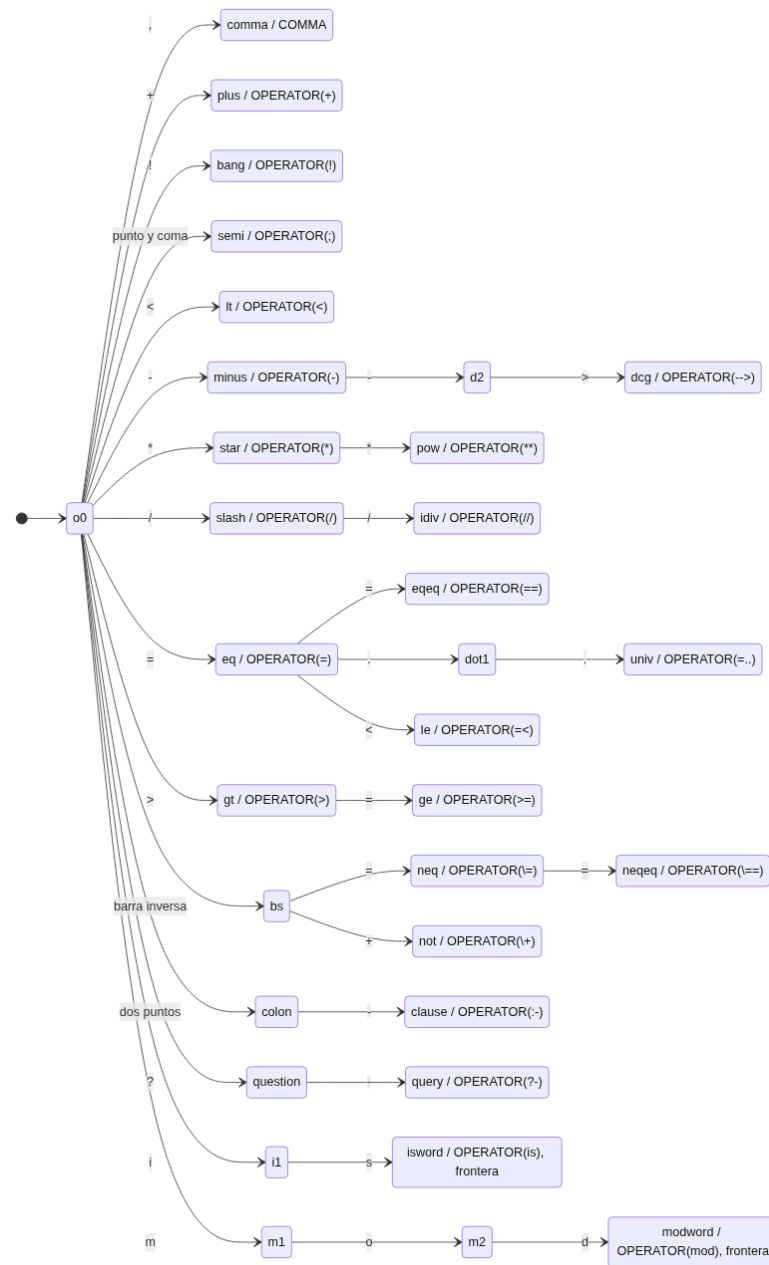


Figura 7. AFN representativo de operadores

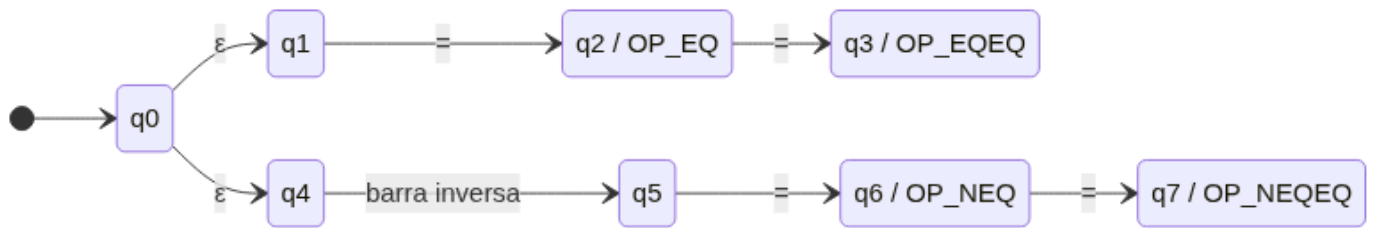


Figura 8. AFD por construcción de subconjuntos

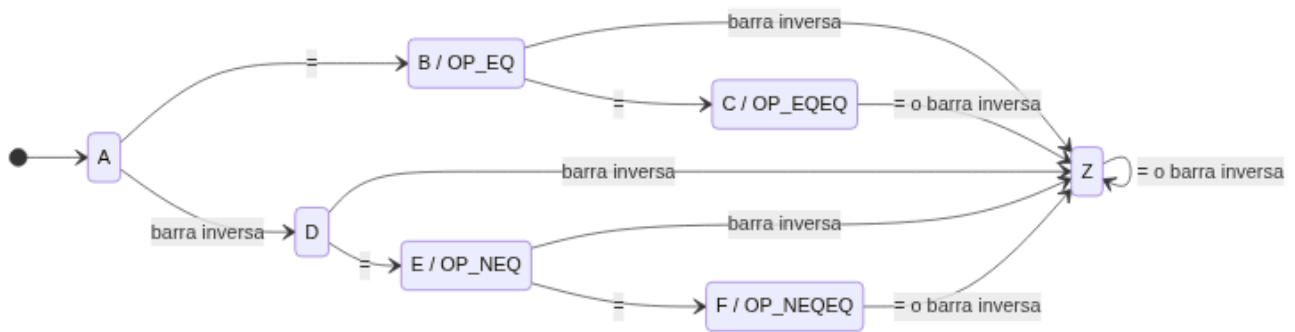


Figura 9. AFD mínimo etiquetado

